

M1 ISPTuningTool

编码规范

命名规则

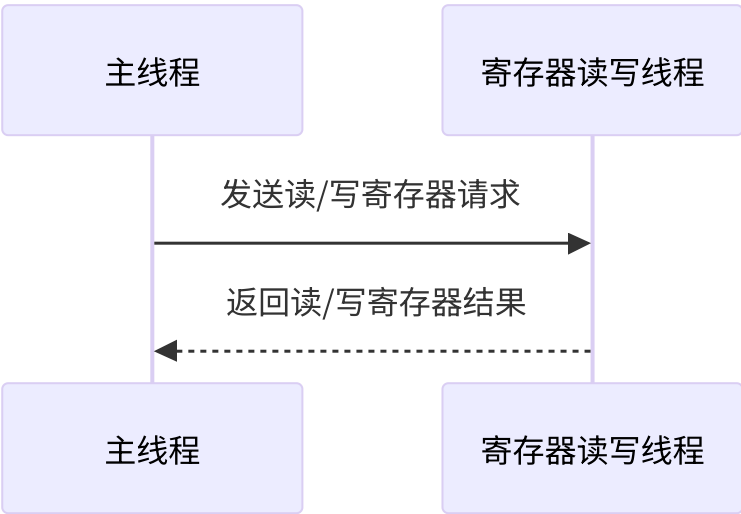
命名规则参照[Qt 官方编码风格指南](#)。

流程图

- 初始化

```
1 start
2 :用户认证（后续讨论）；
3 :打开默认界面；
4 :加载配置；
5 :点击Connect；
6 :检查I2C/SPI是否连通；
7 :加载数据库；
8 :开启寄存器读写线程（存疑）；
9 end
```

- 读写寄存器(考虑写寄存器操作跟生效实时？)



类设计

寄存器读写

- 采用单例实现
指定读写寄存器的SalveID, Address Length, Value Length, Address等

```
1 class QUsbCommunication
2 {
3     - int usbType //指定使用I2C读写还是SPI读写,0:I2c,1:spi
```

```

4      + initUsb()      //初始化, 指定usbType
5      + checkState()   //检查I2C/SPI是否连通
6      + readBurst()
7      + writeBurst()
8
9      - checkI2cState()
10     - checkSpiState()
11     - readI2cBurst()
12     - readSpiBurst()
13     - writeI2cBurst()
14     - writeSpiBurst()
15 }

```

数据库相关

DataBaseOrm

struct

1. Permission (权限结构体)

```

1 struct Permission {
2     std::string permissionName; // 权限名称, 作为主键
3 };

```

作用：表示系统中的权限项

字段说明：

- `permissionName`: 唯一标识一个权限的字符串

对应数据库 `permission` 的内容

2. Register (寄存器结构体)

```

1 struct Register {
2     uint32_t    address;           // 寄存器地址, 作为主键
3     std::string displayName;       // 显示名称
4     std::string rwInfo;           // 读写权限信息
5     int         defaultValue;     // 默认值
6     std::string registerDescription; // 寄存器描述
7     std::string permissionName;   // 关联的权限名称(外键)
8 };

```

作用：描述硬件寄存器的基础信息

字段说明：

- `address`: 寄存器的物理地址(16进制)
- `displayName`: 用于UI显示的寄存器名称
- `rwInfo`: 读写权限, 如"RO"(只读)、"RW"(读写)等
- `defaultValue`: 寄存器的默认值
- `permissionName`: 关联到Permission表的外键

对应数据库 `register` 中的内容

3. Module (模块结构体)

```
1 struct Module {
2     std::string moduleName;           // 模块名称，作为主键
3     std::string moduleDescription;    // 模块描述
4     std::string permissionName;      // 关联的权限名称(外键)
5 };
```

作用：表示硬件或功能模块

字段说明：

- `moduleName`: 模块的唯一标识
- `moduleDescription`: 模块的功能描述
- `permissionName`: 控制访问该模块所需的权限

对应数据库 `Module` 中的内容

4. CombinedRegister (组合寄存器结构体)

```
1 struct CombinedRegister {
2     int combinedId;                   // 组合ID，自增主键
3     std::string moduleName;          // 所属模块名称(外键)
4     std::shared_ptr<int> cameraId;    // 关联的相机ID(可为空)
5     std::string combinedName;        // 组合名称
6     std::string displayName;         // 显示名称
7     std::string combinedDescription; // 组合描述
8     std::string permissionName;      // 关联的权限名称(外键)
9 };
```

作用：将多个寄存器位组合成一个逻辑单元

字段说明：

- `combinedId`: 自动生成的唯一ID
- `moduleName`: 关联的模块名称
- `cameraId`: 使用`shared_ptr`表示可选字段，`null`表示不关联特定相机
- `combinedName`: 组合的逻辑名称
- `displayName`: UI显示用名称
- `permissionName`: 访问该组合寄存器所需的权限

对应数据库 `combined_register` 中的内容

5. RegisterBitInfo (寄存器位信息结构体)

```
1 struct RegisterBitInfo {
2     uint32_t    registerAddr;           // 寄存器地址(外键)
3     int         bitMask;                // 位掩码
4     std::string registerBitName;        // 位名称
5     int         combinedId;             // 所属组合ID(外键)
6     uint32_t    combinedBitMask;        // 在组合寄存器中的位掩码
7     std::string rw;                     // 读写属性
8     int         defaultValue;           // 默认值
9     std::string registerBitDescription; // 位描述
10    std::string permissionName;          // 关联的权限名称(外键)
11 };
```

作用：描述寄存器中各个位域的定义

字段说明：

- `registerAddr`: 关联的寄存器地址
- `bitMask`: 该位在寄存器中的掩码(如0x01表示bit0)
- `registerBitName`: 位域的名称
- `combinedId`: 所属的组合寄存器ID
- `combinedBitMask`: 在组合寄存器中的位掩码
- `rw`: 位的读写属性
- `defaultValue`: 位的默认值
- `permissionName`: 访问该位所需的权限

对应数据库 `register_bit_info` 中的内容

6. UiModule (UI模块结构体)

```
1 struct UiModule {
2     std::string moduleName;             // UI模块名称，作为主键
3     std::string permissionName;         // 关联的权限名称(外键)
4 };
```

作用：表示UI上的功能模块

字段说明：

- `moduleName`: UI模块的唯一标识
- `permissionName`: 访问该UI模块所需的权限

对应数据库 `ui_module` 中的内容

7. UiInfo (UI信息结构体)

```
1 struct UiInfo {
2     std::string      moduleName;      // UI模块名称
3     std::string      controlName;     // 控件名称
4     int              index;           // 控件索引
5     std::shared_ptr<std::string> combinedName; // 关联的组合寄存器名(可为空)
6     std::shared_ptr<uint32_t> registerAddr; // 直接关联的寄存器地址(可为空)
7     std::string      permissionName;  // 关联的权限名称(外键)
8 };
```

作用：描述UI控件与寄存器/组合寄存器的映射关系

字段说明：

- `moduleName`: 所属UI模块名称
- `controlName`: 控件名称
- `index`: 同类型控件的索引号
- `combinedName`: 关联的组合寄存器名称(使用shared_ptr表示可选)
- `registerAddr`: 直接关联的寄存器地址(使用shared_ptr表示可选)
- `permissionName`: 访问该控件所需的权限

对应数据库 `ui_info` 中的内容

8. UiInfoRegModule (UI信息与寄存器模块关联结构体)

```
1 struct UiInfoRegModule {
2     std::string uiModuleName;      // UI模块名称
3     std::string uiControlName;     // UI控件名称
4     std::string regModuleName;     // 关联的寄存器模块名称
5     bool        hasCameraId;       // 是否与相机ID关联
6 };
```

作用：建立UI控件与寄存器模块的关联关系

字段说明：

- `uiModuleName`: UI模块名称
- `uiControlName`: UI控件名称
- `regModuleName`: 关联的寄存器模块名称
- `hasCameraId`: 指示该关联是否与特定相机ID相关

对应数据库 `ui_info_regmodule` 中的内容

9. AWBCalibrationRegister (AWB校准寄存器结构体)

```
1 struct AWBCalibrationRegister {
2     int registerCode;      // 寄存器代码，作为主键
3     std::string moduleName; // 模块名称
4     std::string combinedName; // 组合寄存器名称
5 };
```

作用： 存储自动白平衡(AWB)校准的寄存器配置

字段说明：

- `registerCode` : AWB寄存器的唯一代码
- `moduleName` : 所属模块名称
- `combinedName` : 关联的组合寄存器名称

对应数据库 `awb_calibration_register` 中的内容

结构体关系说明

1. 核心关系：

- `Register` 和 `RegisterBitInfo`：一对多，一个寄存器包含多个位定义
- `Module` 和 `CombinedRegister`：一对多，一个模块包含多个组合寄存器
- `CombinedRegister` 和 `RegisterBitInfo`：一对多，一个组合寄存器包含多个位定义

2. UI映射关系：

- `UiInfo` 通过 `combinedName` 或 `registerAddr` 关联到寄存器系统
- `UiInfoRegModule` 建立UI控件与寄存器模块的间接关联

3. 权限控制：

所有主要结构体都包含 `permissionName` 字段，实现统一的权限控制

4. 相机关联：

- `CombinedRegister` 和 `ModuleBaseAddr` 通过 `cameraId` 关联到特定相机
- `UiInfoRegModule` 通过 `hasCameraId` 指示是否需要考虑相机关联

这些结构体共同构成了一个完整的硬件寄存器管理系统，支持从底层寄存器到位域、再到UI控件的完整映射关系。

functions

1. 初始化数据库

1.1 initDatabase

```
1  /**
2   * @brief 初始化数据库连接
3   * @param databasePath 数据库文件路径
4   *
5   * 示例：
6   * DatabaseOrm::initDatabase("/path/to/database.db");
7   *
8   * 注意： 必须在调用其他数据库操作前执行
9   */
10 void DatabaseOrm::initDatabase(std::string databasePath);
```

使用DatabaseAccess进行数据库的初始化，调用initStorage加载数据库

2. 权限相关接口

2.1 getAllPermissions

```
1  /**
2   * @brief 获取所有权限记录
3   * @return 包含所有Permission对象的vector
4   *
5   * 示例:
6   * auto permissions = DatabaseOrm::getAllPermissions();
7   * for (const auto& perm : permissions) {
8   *     std::cout << "Permission: " << perm.permissionName << std::endl;
9   * }
10  */
11  std::vector<Permission> DatabaseOrm::getAllPermissions();
```

3. 寄存器相关接口

3.1 getAllRegisters

```
1  /**
2   * @brief 获取所有寄存器记录
3   * @return 包含所有Register对象的vector
4   *
5   * 示例:
6   * auto registers = DatabaseOrm::getAllRegisters();
7   * for (const auto& reg : registers) {
8   *     std::cout << "Register: " << reg.displayName
9   *               << " Addr: 0x" << std::hex << reg.address << std::endl;
10  * }
11  */
12  std::vector<Register> DatabaseOrm::getAllRegisters();
```

3.2 getRegistersByCameraId

```
1  /**
2   * @brief 根据相机ID获取关联的寄存器
3   * @param cameraId 相机ID的shared_ptr, 传nullptr表示获取未关联相机的寄存器
4   * @return 符合条件的Register对象vector
5   *
6   * 示例1 - 获取特定相机的寄存器:
7   * auto cameraId = std::make_shared<int>(1);
8   * auto registers = DatabaseOrm::getRegistersByCameraId(cameraId);
9   *
10  * 示例2 - 获取未关联相机的寄存器:
11  * auto registers = DatabaseOrm::getRegistersByCameraId(nullptr);
12  */
13  std::vector<Register> DatabaseOrm::getRegistersByCameraId(std::shared_ptr<int>
    cameraId);
```

3.3 getRegisterByAddress

```

1  /**
2   * @brief 根据寄存器地址获取单个寄存器
3   * @param address 寄存器地址
4   * @return Register对象，如果未找到则返回空对象
5   *
6   * 示例：
7   * auto reg = DatabaseOrm::getRegisterByAddress(0x1234);
8   * if (reg.address != 0) {
9   *     std::cout << "Found register: " << reg.displayName << std::endl;
10  * }
11  */
12 Register DatabaseOrm::getRegisterByAddress(uint32_t address);

```

4. 模块相关接口

4.1 getAllModules

```

1  /**
2   * @brief 获取所有模块记录
3   * @return 包含所有Module对象的vector
4   *
5   * 示例：
6   * auto modules = DatabaseOrm::getAllModules();
7   * for (const auto& mod : modules) {
8   *     std::cout << "Module: " << mod.moduleName << std::endl;
9   * }
10  */
11 std::vector<Module> DatabaseOrm::getAllModules();

```

4.2 getModuleByModuleName

```

1  /**
2   * @brief 根据模块名称获取模块
3   * @param moduleName 模块名称
4   * @return Module对象，如果未找到则返回空对象
5   *
6   * 示例：
7   * auto module = DatabaseOrm::getModuleByModuleName("sensor");
8   * if (!module.moduleName.empty()) {
9   *     std::cout << "Module desc: " << module.moduleDescription << std::endl;
10  * }
11  */
12 Module DatabaseOrm::getModuleByModuleName(std::string moduleName);

```

5. 组合寄存器相关接口

5.1 getCombinedRegistersByModuleNameAndCameraId

```

1  /**
2   * @brief 根据模块名和相机ID获取组合寄存器
3   * @param moduleName 模块名称
4   * @param cameraId 相机ID的shared_ptr，传nullptr表示获取未关联相机的组合寄存器

```

```

5  * @return 符合条件的CombinedRegister对象vector
6  *
7  * 示例1 - 获取特定模块和相机的组合寄存器：
8  * auto cameraId = std::make_shared<int>(1);
9  * auto combinedRegs = DatabaseOrm::getCombinedRegistersByModuleNameAndCameraId("isp",
combinedRegs);
10 *
11 * 示例2 - 获取未关联相机的组合寄存器：
12 * auto combinedRegs = DatabaseOrm::getCombinedRegistersByModuleNameAndCameraId("isp",
nullptr);
13 */
14 std::vector<CombinedRegister> DatabaseOrm::getCombinedRegistersByModuleNameAndCameraId(
15     std::string moduleName, std::shared_ptr<int> cameraId);

```

6. 寄存器位信息相关接口

6.1 getRegisterBitInfoByRegisterAddressAndBitMask

```

1  /**
2  * @brief 根据寄存器地址和位掩码获取位信息
3  * @param regAddress 寄存器地址
4  * @param bitMask 位掩码
5  * @return 符合条件的RegisterBitInfo对象vector
6  *
7  * 示例：
8  * auto bitInfos = DatabaseOrm::getRegisterBitInfoByRegisterAddressAndBitMask(0x1234,
0xFF);
9  * for (const auto& info : bitInfos) {
10 *     std::cout << "Bit name: " << info.registerBitName << std::endl;
11 * }
12 */
13 std::vector<RegisterBitInfo>
DatabaseOrm::getRegisterBitInfoByRegisterAddressAndBitMask(
14     int regAddress, int bitMask);

```

7. UI相关接口

7.1 getUiInfoByModuleNameAndControlName

```

1  /**
2  * @brief 根据UI模块名和控件名获取UI信息
3  * @param moduleName UI模块名
4  * @param controlName 控件名
5  * @return 按index排序的UiInfo对象vector
6  *
7  * 示例：
8  * auto uiInfos = DatabaseOrm::getUiInfoByModuleNameAndControlName("settings",
"exposure");
9  * for (const auto& info : uiInfos) {
10 *     std::cout << "Control " << info.controlName
11 *         << " index: " << info.index << std::endl;
12 * }
13 */

```

```

14 std::vector<UiInfo> DatabaseOrm::getUiInfoByModuleNameAndControlName(
15     std::string moduleName, std::string controlName);

```

8. AWB校准相关接口

8.1 getAWBRegisterCombinedId

```

1  /**
2   * @brief 获取AWB校准的组合寄存器ID
3   * @param moduleName 模块名
4   * @param combinedName 组合寄存器名
5   * @param cameraId 相机ID, 小于0表示不限制相机ID
6   * @return 符合条件的CombinedRegister对象vector
7   *
8   * 示例1 - 获取特定相机的AWB组合寄存器:
9   * auto combinedRegs = DatabaseOrm::getAWBRegisterCombinedId("awb", "calibration", 1);
10  *
11  * 示例2 - 获取所有相机的AWB组合寄存器:
12  * auto combinedRegs = DatabaseOrm::getAWBRegisterCombinedId("awb", "calibration", -1);
13  */
14 std::vector<CombinedRegister> DatabaseOrm::getAWBRegisterCombinedId(
15     std::string moduleName, std::string combinedName, int cameraId);

```

高级使用示例

示例1: 获取模块的所有寄存器及其位信息

```

1  auto moduleName = "isp";
2  auto modules = DatabaseOrm::getAllModules();
3  auto moduleIt = std::find_if(modules.begin(), modules.end(),
4      [&](const Module& m) { return m.moduleName == moduleName; });
5
6  if (moduleIt != modules.end()) {
7      auto registers = DatabaseOrm::getRegistersByModuleName(moduleName);
8      for (const auto& reg : registers) {
9          std::cout << "Register: " << reg.displayName << std::endl;
10         auto bitInfos = DatabaseOrm::getRegisterBitInfoByRegisterAddress(reg.address);
11         for (const auto& bit : bitInfos) {
12             std::cout << "    Bit: " << bit.registerBitName
13                 << " Mask: 0x" << std::hex << bit.bitMask << std::endl;
14         }
15     }
16 }

```

示例2: 通过UI控件查找关联的寄存器

```

1  auto uiModule = "settings";
2  auto controlName = "exposure";
3  auto cameraId = std::make_shared<int>(1);
4
5  // 获取UI信息
6  auto uiInfos = DatabaseOrm::getUiInfoByModuleNameAndControlName(uiModule, controlName);

```

```

7
8 for (const auto& uiInfo : uiInfos) {
9     if (uiInfo.combinedName) {
10         // 通过组合名查找
11         auto combinedRegs =
DatabaseOrm::getCombinedRegisterByUiModuleNameAndControlNameAndCameraId(
12             uiModule, controlName, cameraId);
13         for (const auto& combReg : combinedRegs) {
14             auto bitInfos =
DatabaseOrm::getRegisterBitInfoByCombinedRegisterId(combReg.combinedId);
15             // 处理位信息...
16         }
17     } else if (uiInfo.registerAddr) {
18         // 直接通过寄存器地址查找
19         auto reg = DatabaseOrm::getRegisterByAddress(*uiInfo.registerAddr);
20         // 处理寄存器...
21     }
22 }

```

注

UI相关

- 主界面

```

1 class QMainwnd
2 {
3     - QTreeWidget*
4     - QTabWidget*
5     - QChartView*
6     + initwnd()
7 }

```

- Tab界面共通

```

1 class QTabwndComm
2 {
3     - QPushButton* btnRead
4     - QPushButton* btnLoad
5     - QPushButton* btnSave
6     + read()
7     + load()
8     + save()
9 }

```

- 各控件父类

```

1 class QControlBase
2 {
3     - QString labelName
4     - int bitCount

```

```

5     - uint32_t groupValue
6     + init()      //初始化, 指定bitCount与groupValue默认值
7     + setValue()
8     + getValue()
9     + signalValueChanged() //信号 (表示控件的值发生了变化)
10    + recvValueChanged() //槽 (表示该控件的值需要对应改变)
11
12    + setSensorValue()
13    + getSensorValue()
14    - readSensor()
15    - wrtieSensor()
16
17    //获取控件绑定寄存器信息: 模块id, CameraNo, 寄存器名字等
18 }

```

```

1 //以下接口需要子类实现
2 init()
3 setValue()
4 getValue()
5 recvValueChanged()
6
7 //为外部调用, 实现读/写寄存器的同时更新各控件的value
8 setValueSensor()
9 getValueSensor()
10
11 //为内部使用函数, 子类无需实现
12 readSensor()
13 wrtieSensor()

```

- 由多个checkbox拼成的组合控件

```

1 class QBitCheckBoxGroup
2 {
3     - int bitCount
4     - uint32_t groupValue
5     + init()      //初始化, 指定labelName, bitCount, groupValue默认值
6     + setValue()
7     + getValue()
8     + signalValueChanged() //信号
9     + recvValueChanged() //槽
10 }

```

- 由一个Spin Box, 一个Slider拼成的组合控件

```

1 class QSpinsSliderGroup
2 {
3     - uint32_t groupValue
4     + init()      //初始化, 指定groupValue默认值
5     + setValue()
6     + getValue()
7     + signalValueChange() //信号
8     + recvValueChanged()  //槽
9 }

```

- 由Label标签 + QSpinsSliderGroup + QBitCheckBoxGroup

```

1 class QLabelSpinsSliderCheckBoxGroup
2 {
3     - QString labelName
4     - int bitCount
5     - uint32_t groupValue
6     - QLabel*
7     - QSpinsSliderGroup*
8     - QSpinsSliderGroup*
9     + init()      //初始化, 指定labelName, bitCount, groupValue默认值
10    + setValue()
11    + getValue()
12    + signalValueChange() //信号
13    + recvValueChanged()  //槽
14 }

```

- 由多个QPushButton拼成的组合控件, 设置三态样式应用于ISP各模块开关, 上下左右复合按钮。

```

1 class QPushButtonGroup
2 {
3     - int bitCount
4     - uint32_t groupValue
5     + init()      //初始化, 指定labelName, bitCount, groupValue默认值
6     + setValue()
7     + getValue()
8     + signalValueChange() //信号
9     + recvValueChanged()  //槽
10 }

```

- 由QChart+QLineSeries组成的图表绘制Gamma曲线及CCM色温矩阵图

```

1  class QCurveChartGroup
2  {
3      - int bitCount
4      - uint32_t groupValue
5      + init()      //初始化, 指定labelName, bitCount, groupValue默认值
6      + setValue()
7      + getValue()
8      + setLegend()//图例相关
9      + signalValueChange() //信号
10     + recvValueChanged() //槽
11 }

```

- 由Label标签 + QComboBox下拉框组成

```

1  class QLabelQComboBoxGroup
2  {
3      - QString labelName
4      - int bitCount
5      - uint32_t groupValue
6      - QLabel*
7      - QComboBox*
8      + init()      //初始化, 指定labelName, bitCount, groupValue默认值
9      + setValue()
10     + getValue()
11     + signalValueChange() //信号
12     + recvValueChanged() //槽
13 }

```

- 设置定制窗体类QCustomWindow, 继承QWidget, 其关联的控件如qpushbutton, qcheckbox, qtableview等由配置文件"xxx.qss"读取。

```

1  class QCustomwindow: public QWidget
2  {
3      Q_OBJECT//作为任意功能窗口的父类, 引入控件样式: class childwidget: public Customwindow{ };
4  public:
5      explicit QCustomWindow(QWidget* parent = nullptr, Qt::WindowFlags f =
Qt::WindowFlags());
6      void setTitle(const QString& title);
7      ~QCustomWindow();
8  protected:
9      void paintEvent(QPaintEvent *event);
10
11  private :
12      void onMousePress(QMouseEvent *event);//重写鼠标键盘事件
13
14      void mousePressEvent(QMouseEvent *event);
15
16      void mouseReleaseEvent(QMouseEvent *event);
17
18      void mouseMoveEvent(QMouseEvent *event);
19

```

```
20 void paintTitle(QPainter &painter);  
21 }
```